Announcements 📣

- Project Phase #2 Project Proposal is due this week Saturday at 11:59 PM
- Individual assignment 1 released due end this week Saturday at 11:59 PM
- Quiz 1 next Monday

Web Engineering & Development (SWE 363)

JavaScript Fundamentals II

In today's lecture:

- Objects
- Maps
- String
- Date
- Math
- Exception Handling
- Regular Expressions

Reference:

- Zybook: 4.9 to 4.15

JavaScript Objects

Key Concepts:

- **Object Creation**: Literal notation vs constructor
- **Properties**: Accessing and modifying object properties
- **Methods**: Functions as object properties
- `this` **keyword**: Context and binding
- **Object iteration**: `for...in` loops
- **Object methods**: `Object.keys()`, `Object.values()`, `Object.entries()`

Objects - Creation & Access

```
// Object literal notation
const student = {
    name: "Ahmed",
    age: 20,
    major: "Computer Science",
    greet: function() {
        return `Hello, I'm ${this.name}`;
    }
};
// Accessing properties
console.log(student.name);          // "Ahmed"
console.log(student["age"]);        // 20
console.log(student.greet());       // "Hello, I'm Ahmed"

// Adding new properties
student.gpa = 3.8;
```

Objects - Methods & `this`

```
const calculator = {
    result: 0,
    add: function(num) {
        this.result += num;
        return this;
    },
    multiply: function(num) {
        this.result *= num;
        return this;
    },
    getResult: function() {
        return this.result;
    }
};
// Method chaining
calculator.add(5).multiply(3).add(2);
console.log(calculator.getResult()); // 17
```

Maps

Key Concepts:

- **Map vs Object**: When to use each
- **Map methods**: `set()`, `get()`, `has()`, `delete()`, `clear()`
- **Map iteration**: `for...of` loops
- **Map size**: `size` property
- **Key types**: Any type can be a key

Maps - Creation & Usage

```
// Creating a Map
const studentGrades = {
    "Ahmed": 95,
    "Sara": 87,
    "Omar": 92
};

// Accessing values
console.log(studentGrades.get("Ahmed")); // 95
console.log(studentGrades.has("Sara"));  // true

// Map size
console.log(studentGrades.size); // 3
```

String Objects

Key Concepts:

- **String methods**: `length`, `charAt()`, `indexOf()`, `substring()`
- **String manipulation**: `toUpperCase()`, `toLowerCase()`, `trim()`
- **String searching**: `includes()`, `startsWith()`, `endsWith()`
- **String splitting**: `split()`, `join()`
- **Template literals**: Backticks and `${}` syntax

String Objects - Basic Methods

```
const text = "  Hello, World!  ";

// Length and character access
console.log(text.length);           // 17
console.log(text.charAt(0));        // " "
console.log(text[0]);               // " " (bracket notation)

// Case conversion
console.log(text.toUpperCase());    // "  HELLO, WORLD!  "
console.log(text.toLowerCase());    // "  hello, world!  "

// Trimming whitespace
console.log(text.trim());           // "Hello, World!"
```

String Objects - Advanced Methods

```
const sentence = "JavaScript is awesome and powerful";

// Finding positions
console.log(sentence.indexOf("is"));        // 11
console.log(sentence.lastIndexOf("a"));     // 20

// Extracting substrings
console.log(sentence.substring(0, 10));     // "JavaScript"
console.log(sentence.slice(-7));            // "powerful"

// Splitting and joining
const words = sentence.split(" ");
console.log(words); // ["JavaScript", "is", "awesome", "and", "powerful"]

const joined = words.join("-");
console.log(joined); // "JavaScript-is-awesome-and-powerful"
```

Date Objects

Key Concepts:

- **Date creation**: `new Date()`, `new Date(year, month, day)`
- **Date methods**: `getFullYear()`, `getMonth()`, `getDate()`, `getDay()`
- **Time methods**: `getHours()`, `getMinutes()`, `getSeconds()`, `getMilliseconds()`
- **Date formatting**: `toDateString()`, `toISOString()`, `toLocaleDateString()`

Date Objects - Creation & Basic Methods

```
// Creating dates
const now = new Date();
const specificDate = new Date(2024, 8, 17); // September 17, 2024
const dateFromString = new Date("2024-09-17");

console.log(now);                    // Current date and time
console.log(specificDate);           // September 17, 2024

// Getting date components
console.log(now.getFullYear());      // 2024
console.log(now.getMonth());         // 8 (September, 0-indexed)
console.log(now.getDate());          // 17
console.log(now.getDay());           // 2 (Tuesday, 0=Sunday)
```

Date Objects - Formatting

```
const date = new Date(2024, 8, 17, 14, 30, 0);

// Date formatting
console.log(date.toDateString());         // "Tue Sep 17 2024"
console.log(date.toISOString());          // "2024-09-17T14:30:00.000Z"
console.log(date.toLocaleDateString());  // "9/17/2024" (locale dependent)
```

Math Objects

Key Concepts:

- **Mathematical constants**: `Math.PI`, `Math.E`
- **Rounding methods**: `Math.round()`, `Math.floor()`, `Math.ceil()`, `Math.trunc()`
- **Power and roots**: `Math.pow()`, `Math.sqrt()`
- **Trigonometric functions**: `Math.sin()`, `Math.cos()`, `Math.tan()`
- **Random numbers**: `Math.random()`
- **Min/Max**: `Math.min()`, `Math.max()`

Math Objects - Constants & Rounding

```
// Mathematical constants
console.log(Math.PI);              // 3.141592653589793
console.log(Math.E);               // 2.718281828459045

// Rounding methods
const num = 4.7;
console.log(Math.round(num));      // 5
console.log(Math.floor(num));      // 4
console.log(Math.ceil(num));       // 5
console.log(Math.trunc(num));      // 4

// Rounding to decimal places
const price = 19.99;
const rounded = Math.round(price * 100) / 100;
console.log(rounded);              // 19.99
```

Math Objects - Power, Roots & Random

```
// Power and roots
console.log(Math.pow(2, 3));        // 8
console.log(Math.sqrt(16));         // 4

// Random numbers
console.log(Math.random());         // Random number between 0 and 1

// Random integer between min and max
function randomInt(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomInt(1, 10));      // Random integer between 1 and 10

// Min and Max
console.log(Math.min(5, 2, 8, 1)); // 1
console.log(Math.max(5, 2, 8, 1)); // 8
```

Exception Handling

Key Concepts:

- **try-catch-finally blocks**: Error handling structure
- **Throwing errors**: `throw` statement

Exception Handling - Basic Structure

```
try {
    const result = 10 / 0;
    console.log(result);
} catch (error) {
    console.log("An error occurred:", error.message);
}
function divide(a, b) {
    try {
        if (b === 0) { throw new Error("Division by zero is not allowed"); }
        return a / b;
    } catch (error) {
        console.log("Error:", error.message);
        return null;
    } finally {
        console.log("Division operation completed");
    }
}
```

Regular Expressions

Key Concepts:

- **Regex syntax**: Patterns, flags, and metacharacters
- **Character classes**: `[a-z]`, `[0-9]`, `[^abc]`
- **Quantifiers**: `*`, `+`, `?`, `{n}`, `{n,m}`
- **Anchors**: `^`, `$`, `\b`
- **Special characters**: `\d`, `\w`, `\s`, `.`
- **Regex methods**: `exec()`, `match()`, `replace()`

Regular Expressions - Basic Patterns

```
// Creating regex
const pattern1 = /hello/i;
const pattern2 = new RegExp("hello");

// Basic matching
const text = "Hello, world!";
console.log(pattern1.test(text));        // true
console.log(/world/.test(text));         // true

// Case insensitive flag
const caseInsensitive = /hello/i;
console.log(caseInsensitive.test("HELLO")); // true

// Global flag
const globalPattern = /o/g;
console.log(text.match(globalPattern));  // ["o", "o"]
```

Regular Expressions - Char. Classes & Quantifiers

```
const text = "Contact us at info@example.com or call 123-456-7890";

// Character classes
const emailPattern = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/;
console.log(emailPattern.test(text));    // true

// Quantifiers
const phonePattern = /\d{3}-\d{3}-\d{4}/;
console.log(phonePattern.test(text));    // true

// Word boundaries
const wordPattern = /\b\w{4}\b/g;
console.log(text.match(wordPattern));    // ["info", "call"]

// Special characters
const digitPattern = /\d+/g;
console.log(text.match(digitPattern));   // ["123", "456", "7890"]
```

Regular Expressions - Practical Examples

```
// Email validation
function isValidEmail(email) {
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return emailRegex.test(email);
}

console.log(isValidEmail("user@example.com"));     // true
console.log(isValidEmail("invalid-email"));        // false

// Phone number formatting
function formatPhone(phone) {
    const phoneRegex = /(\d{3})(\d{3})(\d{4})/;
    return phone.replace(phoneRegex, "($1) $2-$3");
}

console.log(formatPhone("1234567890"));            // "(123) 456-7890"
```

30m

Demo

4.2 JS Advanved

Next Classes

- Proposal review session
- Quiz on HTML & CSS & JS